

1 — Qu'est-ce que la containerisation (simple)

- Container = processus isolé sur un noyau Linux partagé.
- Chaque container contient l'application + ses dépendances (bibliothèques, runtime) — mais pas un noyau complet.
- Avantages : démarrage très rapide, faible empreinte mémoire/stockage, images reproductibles, facile à déployer et scaler.

2 — Archive / image de conteneur vs machine virtuelle (VM)

Image / archive (Docker / OCI)

- Image = pile de couches immuables (layered filesystem) + métadonnées (commandes, ports, variables).
- Format moderne : OCI image (standard), stockée localement en tant que couches (ex. `/var/lib/docker`).
- Petite taille (couches partagées entre images), démarrage en ms-s.

Archive (tar, zip)

- Fichier compressé contenant fichiers d'application. Utile pour distribution, mais pas exécutable tel quel comme conteneur (il faut extraire et configurer l'environnement).

VM (Virtual Machine)

- VM = image disque complète (qcow2, VMDK, VHD) + hyperviseur qui émule matériel.

- Contient un noyau + OS complet => plus lourd (GB), démarrage lent (sec–min), meilleure isolation (interopérable avec différents noyaux).
- Cas d'usage : lorsqu'il faut isoler complètement le noyau, supporter plusieurs OS sur le même host, ou besoins de sécurité renforcée.

Comparaison synthétique

- Isolation: VM > Container
- Démarrage: Container >> VM
- Taille stockage: Container << VM
- Partage de ressources: Containers partagent noyau → plus efficace
- Cas d'usage: Dev/test + microservices → containers; hébergement multi-OS ou sécurité forte → VM.

3 — Hyperviseur / container runtime (architecture)

- Hyperviseur Type 1 (bare metal) : ex. VMware ESXi, Hyper-V, KVM sur bare metal — exécute VMs directement sur matériel.
- Hyperviseur Type 2 : ex. VirtualBox — tourne au-dessus d'un OS hôte.
- Container runtime : ex. containerd, runc, CRI-O. Docker utilise **containerd/runc** pour lancer des conteneurs (pas d'hyperviseur).
- Isolation Linux : namespaces (PID, NET, MNT, IPC, UTS, USER) + cgroups (contrôle CPU/mémoire/IO) + SELinux/AppArmor pour sécurité.

4 — Structure de fichiers Linux importante pour Docker

- `/var/lib/docker` : stockage local des images, conteneurs, volumes (selon config).
- `/etc/docker/daemon.json` : config du daemon Docker.
- `/var/run/docker.sock` : socket Unix pour contrôler le daemon Docker (attention sécurité : accès = contrôle total).
- Points système généraux : `/etc` (configs), `/var/log` (logs — conteneurs peuvent écrire ici via volumes), `/usr/bin` (binaries), `/home` (utilisateurs).
- Comprendre le UnionFS (overlayfs) : les couches d'images sont empilées, seule la couche supérieure est modifiable pour un conteneur.

5 — Images Docker : construction et concepts

- Dockerfile → instructions pour construire une image (FROM, RUN, COPY, CMD, EXPOSE, VOLUME, WORKDIR...).
- Multistage build : construire dans une étape puis copier l'artefact final dans une image plus petite (ex. builder → final).
- Layers : chaque instruction crée un layer ; réutilisation accélère builds.
- Tagging & Registry : `repo:tag` (ex. `myrepo/myapp:1.0`). Push/pull vers Docker Hub / registry privée.
- Best practice : images petites (alpine, distroless), ignorer fichiers inutiles via `.dockerignore`, utiliser utilisateurs non-root, définir `HEALTHCHECK`.

6 — Volumes, réseau, et sécurité

- Volumes : persistantes, indépendantes du cycle de vie du conteneur (`docker volume create, -v /host/path:/container/path`).
- Bind-mounts : monter dossier hôte dans conteneur (dev).
- Réseaux : bridge (par défaut), host, overlay (pour swarm/k8s). `docker network create`.
- Sécurité : limiter capabilities, user namespaces, AppArmor/SELinux, scanner images (Trivy), ne pas exposer `/var/run/docker.sock`.

7 — Commandes Docker — cheat-sheet hands-on (regroupe et complète ton listing)

Commandes essentielles :

```
# status
systemctl status docker
```

```
# hello
docker run hello-world
```

```
# images & containers
docker images
docker ps          # running
docker ps -a       # all
```

```
# run
docker run --name web01 -d -p 9080:80 nginx
docker logs web01
docker inspect web01
```

```
# network / ip
ip addr show
docker exec -it web01 /bin/sh    # shell dans conteneur
```

```
docker exec -it web01 /bin/bash

# build image
docker build -t tesimg .

# list & stop / rm
docker stop web01
docker rm web01
docker rm $(docker ps -a -q)      # remove all containers
docker rmi <imageID>             # delete image
docker system prune -a           # cleanup unused

# volumes & networks
docker volume ls
docker volume rm <vol>
docker network ls
docker network create mynet

# docker-compose (si installé)
docker compose up -d
docker compose down
```

8 — Exemple Dockerfile (ton exemple, commenté & amélioré)

Ton Dockerfile multistage (commenté) :

```
# étape de build : récupérer template et targer un archive
FROM ubuntu:latest AS BUILD_IMAGE
RUN apt update && apt install -y wget unzip
RUN wget
https://www.tooplate.com/zip-templates/2128_tween_agency.zip
RUN unzip 2128_tween_agency.zip && cd 2128_tween_agency && tar
-czf tween.tgz * && mv tween.tgz /root/tween.tgz

# image finale
```

```
FROM ubuntu:latest
LABEL "project"="Marketing"
ENV DEBIAN_FRONTEND=noninteractive

RUN apt update && apt install -y apache2 git wget
COPY --from=BUILD_IMAGE /root/tween.tgz /var/www/html/
RUN cd /var/www/html/ && tar xzf tween.tgz

VOLUME /var/log/apache2
WORKDIR /var/www/html/
EXPOSE 80
CMD ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]
```

Points d'amélioration :

- préférer `alpine` ou `debian:slim` si possible pour réduire taille.
- nettoyer le cache apt (`apt-get clean && rm -rf /var/lib/apt/lists/*`) dans la même RUN pour réduire la taille du layer.
- ajouter `HEALTHCHECK` pour que l'orchestrateur sache si le container est vivant.

9 — Hands-on pas à pas (décrit exactement ce que tu peux taper)

1. Vérifier le service Docker :

```
systemctl status docker
```

2. Test rapide :

```
docker run hello-world
```

3. Lancer un container Nginx exemple :

```
docker run --name web01 -d -p 9080:80 nginx
docker ps
docker logs web01
docker inspect web01
# depuis l'hôte : curl localhost:9080
curl http://127.0.0.1:9080
```

4. Construire ton image (Dockerfile présent dans `images/`) :

```
cd images
docker build -t tesimg .
docker images
docker run -P -d --name tesimg_run tesimg
docker ps
# trouver IP du container (exemple)
docker inspect -f '{{range
.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' tesimg_run
```

5. Nettoyage :

```
docker stop web01 tesimg_run
docker rm web01 tesimg_run
docker rmi tesimg
docker system prune -a # supprime images non utilisées /
containers / networks
```

10 — Exemple d'architecture microservices (cas pratique)

Supposons un projet vprofil composé de :

- **auth** (service d'authentification) — Node/Express
- **api** (back-end) — Java/SpringBoot
- **frontend** (SPA) — Nginx servant fichiers statiques
- **db** — Postgres

Exemple minimal **docker-compose.yml**:

```
version: "3.8"
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: vprofil
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: vprofil
    volumes:
      - db_data:/var/lib/postgresql/data

  auth:
    build: ./auth
    ports:
      - "8081:8081"
    environment:
      DATABASE_URL: postgres://vprofil:secret@db:5432/vprofil
    depends_on:
      - db

  api:
    build: ./api
    ports:
      - "8080:8080"
    environment:
      AUTH_URL: http://auth:8081
      DATABASE_URL: postgres://vprofil:secret@db:5432/vprofil
    depends_on:
```



```
- db
- auth

frontend:
  build: ./frontend
  ports:
    - "80:80"
  depends_on:
    - api
volumes:
  db_data:
```

Déployer :

```
docker compose up -d --build
docker compose ps
docker compose logs -f api
```

Avantages :

- chaque service isolé, scalable indépendamment (`docker compose up --scale api=3`);
- réseaux internes gérés automatiquement (`docker compose` crée un réseau bridge).

11 — Intégration avec Vagrant (ton Vagrantfile)

- Vagrant + VirtualBox peut être utile pour fournir un VM reproductible (ex. pour dev Windows/Mac) et installer Docker dedans (ton Vagrantfile provisionne Docker apt repo).
- Avantage : tu gardes l'environnement Docker isolé de ton poste, utile si tu veux reproduire CI local.

- Inconvénient : double abstraction (VM + container) => overhead. Pour prod, préférer host Linux + Docker natif.

12 — Bonnes pratiques pour containeriser un projet réel

- Utilise multistage builds pour réduire taille.
- Évite de lancer un processus supervisé (systemd) — un container = 1 processus principal bien défini.
- Utilise `.dockerignore` pour exclure `node_modules`, `.git`, fichiers temporaires.
- Ne stocke pas secrets dans `ENV` dans l'image — utilise secrets managers, variables d'environnement du runtime, ou Docker secrets.
- Healthchecks + readiness probes (si orchestrateur).
- Conserve logs écrits vers `stdout/stderr` pour que Docker les collecte (`docker logs`). Utilise volumes pour persistance (ex. bases, fichiers d'uploads).
- Scanner images (Trivy), utiliser images officielles/minimales.

13 — Monitoring & orchestration (aperçu)

- Pour prod et scale : utiliser Kubernetes, Docker Swarm ou Nomad.
- Monitoring : Prometheus + Grafana, logs : ELK/EFK stack.
- CI/CD : build images dans CI (GitHub Actions, GitLab CI), push vers registry, déployer via helm/compose.

14 — Sécurité pratique (rapide)

- Scanner images, limiter capabilities, user namespaces, ne pas monter docker.sock inutilement.
- Mettre firewall, limiter ports exposés, utiliser TLS pour registries.

15 — Résumé / plan d'action pour un atelier pratique (1 heure)

1. Installer Docker (instructions Ubuntu fournies dans ta note).
2. `docker run hello-world` → vérifier.
3. Lancer `nginx` exemple (`docker run -d -p 9080:80 nginx`) -> voir dans navigateur.
4. Construire l'image from Dockerfile (`docker build -t tesimg .`) et lancer.
5. Déployer un petit stack via `docker compose` (auth/api/frontend/db).
6. Nettoyage & sauvegarde images.
7. Bonus : convertir ce lab en Vagrantfile pour que les étudiants répliquent localement.